

STUDENT 1

Environment and LLM Observation Builder

Make the cyber world understandable to the LLM and safely apply the action it chooses.

ABMS-LLM Cyber Organizational Simulation

Beginner work plan for implementing LLM agents in the current repository. This document is written for both the assigned student and an AI coding assistant.

Main research question

How does agent behavior and performance differ when LLM-driven cyber agents are organized as an independent swarm versus a centralized hierarchy?

Important decision

- LLM cognition is now the main implementation target.
- The existing rule-based cognition stays small and working only as a baseline and API-failure fallback.
- Do not delete the rule baseline, because the research needs a control and developers need a cheap debugging mode.
- Work in the Conda environment named mesa and use Mesa 3.x APIs.

1. Your Job in Simple Words

You own the network, nodes, vulnerabilities, action rules and the data sent to an LLM. You do not call the LLM API yourself. Your output is a clean observation dictionary; your input from Student 2 is a validated action.

How one LLM decision works

- 1 Student 1 turns the current simulation state into a small observation dictionary.
- 2 Student 2 sends that observation to the LLM with instructions and allowed actions.
- 3 The LLM returns one JSON decision. Student 2 checks it before trusting it.
- 4 Student 1's resolver applies the valid action and returns the result.
- 5 Student 3 records behavior, outcome, tokens, time, errors and estimated cost.
- 6 Student 4 shows safe decision summaries and metrics in the UI and documentation.

Words you need to know

Word	Simple meaning
LLM	A language model that reads an observation and chooses an action.
API	The online service used by Python to ask the LLM for a response.
Prompt	The instructions and simulation information sent to the LLM.
JSON	A strict text format used so Python can safely read the LLM answer.
Token	A small unit of text used to measure API usage and cost.
Fallback	A safe rule-based action used when the API or LLM answer fails.
Mock client	A fake API used in tests so tests are fast, free and repeatable.

2. Files You Own

File	What it should do
abms_cyber/config/default_config.py	Add environment and safe LLM-related limits agreed with the team.
abms_cyber/environment/node.py	Store each machine's state.
abms_cyber/environment/network_graph.py	Create the graph, nodes, links, start nodes and target.
abms_cyber/environment/action_resolver.py	Create this file to check and apply actions.
abms_cyber/environment/observation.py	Create this file to convert model state into simple JSON-ready data.
abms_cyber/environment/detection.py	Create or keep a small helper for detection changes.

What you must build

- A function such as `build_observation(agent, model)` that returns only strings, numbers, booleans, lists and dictionaries.
- A clear list of allowed actions and valid target ids. The LLM must never invent a new action and execute it directly.
- An action resolver that checks MOVE, SCAN, EXPLOIT and ESCALATE before changing the world.
- Seeded network generation so the same experiment seed creates the same starting world.
- Tests for valid actions, invalid actions, detection changes and observation fields.

Stay inside your role

Do not redesign another student's layer without agreement. If you need a new field or function, first write a small example of the input and output, then ask its owner to approve the interface.

3. Shared LLM Data Format

All students should use one simple data format. Student 1 builds the observation. Student 2 sends it to the LLM and validates the response. Students 3 and 4 read safe metadata from the result.

Example observation

```
{
  "agent_id": 1,
  "organization": "swarm",
  "current_node": "3",
  "privilege_level": 0,
  "known_nodes": ["3", "4"],
  "recent_actions": ["SCAN:3"],
  "detection_level": 0.20,
  "goal": "Compromise the target with admin privilege",
  "allowed_actions": ["SCAN", "MOVE", "EXPLOIT", "ESCALATE"]
}
```

Example LLM response

```
{
  "action": "EXPLOIT",
  "target_id": "3",
  "reason": "The current node is scanned and not compromised."
}
```

Safety rules

- Never execute Python code returned by the LLM.
- Never trust action or target text until it is checked against allowed values.
- Never hardcode, commit, print or display the API key.
- Use environment variables for secrets and fake clients for normal tests.
- Keep prompts small enough to control cost and store only safe logs.

4. Build It Step by Step

- 1 Write down every fact an agent needs for one decision. Keep it short; do not send Python objects or the whole source code.
- 2 Create `observation.py` and return a JSON-ready dictionary. Follow the project's visibility rule: network topology is visible, while each agent still has its own memory/history.
- 3 Include the goal, current position, privilege, detection level, known state and allowed actions.
- 4 Create `action_resolver.py`. It receives a validated action from Student 2 and returns success/failure plus a short reason.
- 5 Move probability and detection rules out of API/prompt code. LLM text must never directly modify a node.
- 6 Use the model's seeded random generator and write small tests with forced success/failure.
- 7 Give Student 2 two sample observations and document every field in plain language.

When using an AI coding assistant

- Give the assistant this PDF and the repository files named in your assignment.
- Ask it to inspect before editing and to explain what already exists.
- Request one small task at a time; review the diff before moving to the next task.
- Require tests with a mock API before making a paid live request.
- Do not accept code that hides errors, exposes secrets or changes unrelated files.

5. Tests and Definition of Done

Your work is done only when these checks pass:

- The observation can be converted with `json.dumps` without an error.
- No password, API key or Python object appears in the observation.
- An unknown action is rejected without changing model state.
- An invalid target id is rejected without crashing.
- Same seed produces the same initial graph and node values.
- Every accepted action produces a clear result that Student 2 can record in memory.

Minimum evidence to submit

- List of files changed and one sentence explaining each file.
- Automated test output for normal, invalid and failure behavior.
- One end-to-end mocked-LLM run that costs nothing.
- One carefully limited live-LLM smoke run if the team has an API key and budget.
- A note listing remaining limitations and any assumptions agreed by the team.

Cost warning

Do not begin with many trials. Start with mocked responses, then one live step, then one short episode. Student 3 must add call/token/cost limits before the team runs large experiments.

6. Six-Week Plan and Team Handoffs

Week	Main result
1	Agree the observation and action JSON formats with Student 2.
2	Create observation.py and tests.
3	Create action_resolver.py and detection rules.
4	Make network randomness reproducible and finish action tests.
5	Integrate with real LLM responses and fix rejected/unclear cases.
6	Document fields, demonstrate behavior and support final experiments.

Required team handoffs

- Student 1 gives Student 2 the observation fields and action-resolver interface.
- Student 2 gives Student 3 sanitized LLM usage events and gives Student 4 safe decision-log fields.
- Student 3 gives Student 4 stable DataCollector column names and termination status.
- Student 4 gives everyone the final runbook and reports integration defects to the correct owner.

Four contracts to agree in Week 1

- Observation JSON fields.
- LLM response/action JSON fields.
- One model-step sequence for swarm and centralized modes.
- LLM usage and decision-log fields for metrics and UI.

7. Prompt to Give an AI Coding Assistant

Attach or provide this PDF and then send the following prompt. The assistant must still inspect the real repository because this plan describes responsibilities, not the final code state.

You are helping Student 1 on AbstractCyberSim. Work only on the environment, observation builder and action resolver. Read Knowledge/Requierment, Knowledge/Important rules, abms_cyber/environment, config/default_config.py and the agent/action interface before editing. The research is LLM-first. Do not add API calls here. Build JSON-ready observations for the LLM and safely validate/apply SCAN, MOVE, EXPLOIT and ESCALATE. Preserve Mesa 3.x, seeded randomness and existing user changes. Explain each change simply, add tests, and run the relevant tests plus one simulation.

Ask the AI to finish each reply with

- What it changed in beginner-friendly language.
- Which files it changed.
- Which tests and commands it ran and their results.
- Any unresolved problem or decision needed from another student.
- Whether any live API call was made and its approximate usage/cost.

Final reminder

The goal is not merely to make an API call. The goal is to create a controlled research system where valid LLM decisions can be compared fairly between swarm and centralized organizations.